A Survey of Recent Advances in VLA

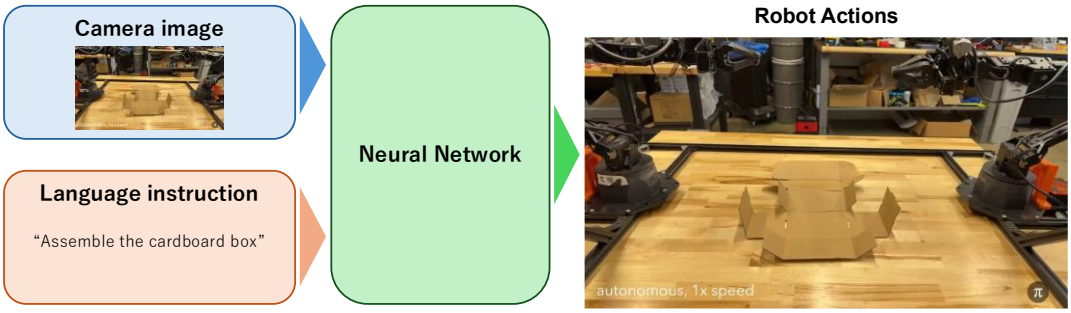
Research Seminar 2026/05/14
Kaneyoshi Hiratsuka

[E] Good afternoon, everyone. Today I will present a survey of three recent papers on Vision–Language–Action models, or VLAs, which are quickly becoming the standard architecture for general-purpose robotic manipulation. Rather than picking three random papers, I have selected one paper for each of the three structural components of a VLA — the encoder, the vision–language model, and the action model — so that we can see how the field is progressing along all three axes at once.

[J] 皆さん、こんにちは。本日は、汎用ロボットマニピュレーションの標準アーキテクチャになりつつある「Vision–Language–Action（VLA）モデル」に関する最新の 3 本の論文を紹介します。3 本の選び方には意図があって、VLA を構成する 3 つのモジュール — エンコーダ、Vision–Language Model（VLM）、Action Model — それぞれを代表する論文を 1 本ずつ選びました。これにより、VLA 分野が 3 つの軸で同時にどう進化しているかを俯瞰できる構成になっています。

What is a VLA Model?

- Inputs: images + natural language
- Output: continuous robot actions
- Goal: generalize across objects, scenes, instructions



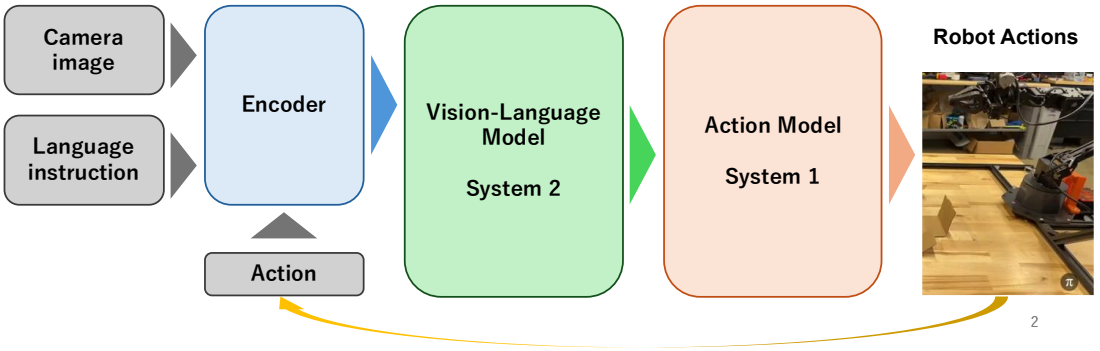
https://www.physicalintelligence.company/blog/pi0

[E] Let me first clarify what a VLA model is. A VLA takes two inputs — visual observations from the robot's cameras, and a natural-language instruction such as "Assemble the cardboard box" — and directly outputs continuous or discrete robot actions, all within a single neural architecture. The dream is to do for robots what large language models did for text: train one big model on diverse data, and have it generalize across objects, environments, and instructions.

[J] まず VLA モデルとは何かを簡単に説明します。VLA は 2 種類の入力を受け取ります。ひとつはロボットカメラからの画像、もうひとつは「リンゴを取ってかごに入れて」のような自然言語の指示です。これを単一のニューラルネットワークに通して、連続値のロボット動作を直接出力します。LLM が「テキストを大量に学習させればあらゆるタスクに汎化する」という発想だったのと同じように、VLA では「多様なロボットデータで大きなモデルを学習すれば、物体・環境・指示に対して汎化するはず」というのが目標です。

Three Building Blocks of a VLA

- Encoder: Raw observation → compact representation
- VLM: “What to do” → semantic & causal reasoning
- Action Model: “How to move” → continuous trajectories



[E] A modern VLA can be decomposed into three building blocks, and this decomposition is the key to today's talk. The Encoder turns raw pixels and proprioceptive states into compact representations. The VLM, which is essentially a vision–language model backbone, performs the high-level semantic reasoning — it decides *what* to do. And finally, the Action Model converts those high-level decisions into smooth, continuous trajectories that the robot's motors can actually execute — it decides *how* to move. The three papers I will present each target exactly one of these blocks, which is why I think they tell a connected story.

[J]現代の VLA は大きく 3 つのブロックに分解できます。これが本日の発表の骨格になります。まず Encoder は、生のピクセルや関節角度などの観測値を、コンパクトな表現に変換します。次に VLM は、いわば Vision–Language モデルのバックボーンで、「何をすべきか」という高レベルの意味的・因果的な推論を担当します。最後に Action Model が、その高レベルの決定を、ロボットモータが実際に追従できる滑らかな連続軌道に変換する、つまり「どう動くか」を決めます。本日紹介する 3 本の論文は、ちょうどこの 3 つのブロックを 1 つずつターゲットにしているので、ばらばらの論文紹介ではなく、ひとつの繋がった物語として聞いていただけると思います。

A Survey of Recent Advances in VLA

1. **UniT: Toward a Unified Physical Language for Human-to-Humanoid Policy Learning and World Modeling**
   B. Chen et al., “UniT: Toward a unified physical language for human-to-humanoid policy learning and world modeling.”
2. **LaST-R1: Reinforcing Robotic Manipulation via Adaptive Physical Latent Reasoning**
   H. Chen et al., “Last-r1: Reinforcing robotic manipulation via adaptive physical latent reasoning,” 2026. [Online]. Available: https://arxiv.org/abs/2604.28192
3. **Streaming Flow Policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories**
   S. Jiang et al., "Streaming Flow Policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories," in 9th Annual Conference on Robot Learning, CoRL 2025, 2025.

[E] Despite rapid progress, the three components each face a hard open problem. On the Encoder side, robot demonstration data is orders of magnitude scarcer than the text used to train LLMs — how do we leverage abundant human video instead? On the VLM side, chain-of-thought reasoning works for text, but how do we extend "thinking in language" to "thinking in the body" with closed-loop physical control? And on the Action Model side, diffusion and flow-matching policies are very expressive, but they suffer from high inference latency. Each of the three papers we will see today tackles exactly one of these challenges: UniT for the encoder, LaST-R1 for the VLM, and Streaming Flow Policy for the action model.

[J]急速な進歩にもかかわらず、3 つのコンポーネントにはそれぞれ大きな未解決問題があります。Encoder 側では、ロボットの実演データは LLM の学習に使われるテキストに比べて何桁も少ない。じゃあ大量に手に入る人間の動画データをどう活用するか、という問題です。VLM 側では、テキストでは Chain-of-Thought が効きますが、これを「身体で考える」、つまり閉ループの物理制御に拡張するにはどうするか。そして Action Model 側では、Diffusion や flow matching ベースのポリシーは表現力は高いものの、推論レイテンシが大きいという問題があります。本日紹介する 3 本は、ちょうどこの 3 つの課題に 1 対 1 で対応しています。エンコーダの UniT、VLM の LaST-R1、Action Model の Streaming Flow Policy です。

A Survey of Recent Advances in VLA

1. **UniT: Toward a Unified Physical Language for Human-to-Humanoid Policy Learning and World Modeling**
   B. Chen et al., “UniT: Toward a unified physical language for human-to-humanoid policy learning and world modeling.”
2. **LaST-R1: Reinforcing Robotic Manipulation via Adaptive Physical Latent Reasoning**
   H. Chen et al., “Last-r1: Reinforcing robotic manipulation via adaptive physical latent reasoning,” 2026. [Online]. Available: https://arxiv.org/abs/2604.28192
3. **Streaming Flow Policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories**
   S. Jiang et al., "Streaming Flow Policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories," in 9th Annual Conference on Robot Learning, CoRL 2025, 2025.

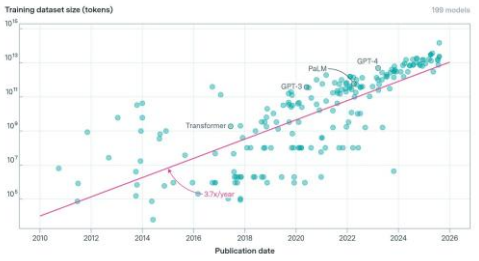
[E] Let's start with the first paper, UniT. The one-line summary is: UniT introduces a latent action tokenizer that bridges humans and humanoids by *anchoring* on visual outcomes. Three keywords to keep in mind: visual anchoring, cross-reconstruction, and embodiment-agnostic.

[J] では、1 本目の UniT から始めます。1 行で言うと、「人間とヒューマノイドの動きを、視覚的な結果に錨を下ろすことで橋渡しする潜在アクション・トークナイザ」です。覚えていただきたいキーワードは 3 つ、Visual Anchoring（視覚への錨）、Cross-Reconstruction（相互再構成）、Embodiment-Agnostic（身体性に依らない表現）です。

Background: Robot Data is Scarce

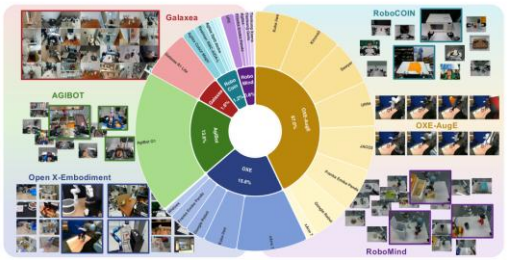
- ✅ Egocentric human videos = abundant, rich physical priors
- ❌ humans ≠ humanoids (DoF, control modes, action params differ)
- ❌ Classical fix: IK-based retargeting → labor-intensive, physically inconsistent

LLM: 10^{13}~ tokens



https://epoch.ai/data-insights/dataset-size-trend

VLA: 10^{6}~10^{7} trajectories



https://arxiv.org/html/2602.11236v2

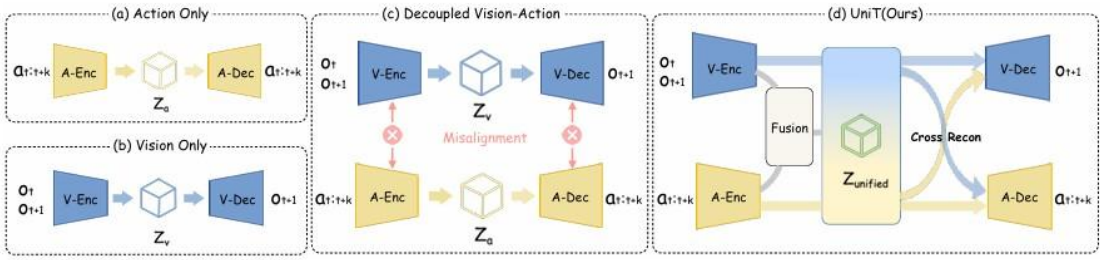
Solution direction: learn a shared latent action space

[E] Why is this paper looking at humans at all? Because LLMs were trained on tens of trillions of tokens, while robot demonstration datasets contain at most a few million trajectories. The gap is enormous. Egocentric human videos, on the other hand, can be collected at massive scale and they contain rich physical interaction prior. The problem is that humans and humanoids differ in degrees of freedom, control modes, and action parameterization. The classical fix is inverse-kinematics retargeting, but that is case-by-case, labor-intensive, and often physically inconsistent. So the recent direction is to absorb embodiment differences at the *representation* level, by learning a shared latent action space.

[J]なぜわざわざ人間に目を向けるのか。LLM が何十兆トークンも学習しているのに対して、ロボットの実演データはせいぜい数百万軌道です。スケールが何桁も違います。一方、一人称視点の人間の動画は大量に集められて、しかも物理的な相互作用のリッチな事前知識を含んでいる。まさに欲しいものなんです。問題は、人間とヒューマノイドでは自由度、制御モード、行動パラメータがどれも違うこと。古典的な解決策は IK によるリターゲティングですが、ケースバイケースで手間がかかり、物理的に矛盾することも多い。そこで最近の方向性は、身体性の違いを「表現のレベル」で吸収しよう、つまり共有の潜在アクション空間を学習しようというものです。

Existing Latent Action Paradigms

- (a) Action-only → ❌ no visual grounding, distribution shift
- (b) Vision-only → ❌ entangled with lighting/textures, loses motor precision
- (c) Decoupled V+A → ❌ disjoint vocabularies, no shared structure
- (d) UniT → ✅ mutually reconstructed, vision as universal anchor



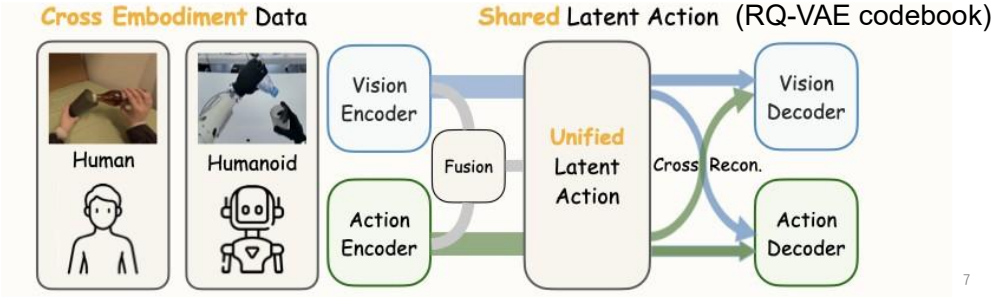
[E] The authors of UniT classify existing latent-action methods into three categories, shown in panels (a), (b), and (c). Action-only methods tokenize only the actions, but without any visual grounding they are vulnerable to distribution shift. Vision-only methods avoid the kinematic mismatch but tend to entangle low-level appearance confounders such as lighting and texture, which hurts motor precision. Decoupled vision–action methods tokenize the two modalities separately, but the two vocabularies end up disjoint and fail to capture shared physical structure. UniT, in panel (d), takes a different stance: vision and action are *mutually reconstructed*, with vision serving as the universal anchor across embodiments.

[J] UniT の著者らは、既存の潜在アクション手法を 3 つに分類しています。図の(a)(b)(c)です。(a)Action-only 法はアクションだけをトークン化しますが、視覚的な接地がないため分布シフトに弱い。(b)Vision-only 法は運動学のミスマッチは回避できるものの、照明やテクスチャといった見た目の余計な要因が絡んでしまい、モータ精度が落ちる。(c)Decoupled 法は視覚とアクションを別々にトークン化しますが、語彙が分離してしまい、両者に共通する物理構造を捉えられない。これに対して UniT は(d)、視覚とアクションを「相互に再構成しあう」というスタンスを取ります。視覚を、身体性をまたぐ普遍的な錨として使うのが要です。

UniT Archtecture

Three encoders, one shared codebook

- Visual branch (input: DINOv2 features o_t, o_{t+k}) → inverse-dynamics latent
- Action branch (input: state s_t, action chunk $a_{t:t+k}$) → motor-control latent
- Fusion branch (V + A) → visuo-motor latent



[E] Now the architecture. UniT takes as input an observation pair at times t and t+k, the current robot state, and the action chunk between them. Three parallel transformer encoders process this information. The visual branch consumes frozen DINOv2 features and behaves like an inverse-dynamics model — it asks "what action explains this visual transition?". The action branch encodes the (state and action chunk) through embodiment-specific MLPs and a shared transformer to produce a motor-control latent. The fusion branch combines both modalities. The continuous outputs of all three branches are then quantized into a *shared* discrete codebook using a Residual Quantization VAE, producing what the authors call Unified Latent Action tokens. Every quantized token is then decoded both by a shared visual decoder and by an embodiment-specific action decoder.

[J]アーキテクチャを見ていきます。UniT は、時刻 t と t+k の観測ペア、現在のロボット状態、その間のアクションチャンクを入力にします。これを 3 つの並列な Transformer エンコーダで処理します。Visual branch は、凍結された DINOv2 の特徴量を入力に取り、逆動力学モデルのように振る舞います。つまり「この見た目の変化を説明するアクションは何か?」を考えるブランチです。Action branch は、状態とアクションチャンクを身体性ごとの MLP と共有 Transformer に通して、モータ制御の潜在表現を作ります。Fusion branch は両方を統合する役目です。3 つのブランチの連続的な出力を、Residual Quantization VAE で「共有の」離散コードブックに量子化します。これが論文の言う Unified Latent Action トークンです。そして各量子化トークンを、共有の Visual Decoder と、身体性ごとの Action Decoder の両方でデコードします。

The key idea: Cross-Reconstruction

- Action features → predict visual outcome
  - ➡ kinematics anchored to pixel-space consequences
- Visual features → reconstruct action
  - ➡ strips away appearance noise (lighting, texture)

$$L = \sum_{i \in \{v,a,m\}} \left[\lambda_v L_{cos}\left(\hat{f}^{(i)}_{t+k}, f_{t+k}\right) + \lambda_a L_{act}\left(\hat{a}^{(i)}_{t:t+k}, a_{t:t+k}\right) \right] + L_{RQ}$$

cosine similarity loss for visual feature reconstruction

action reconstruction loss

RQ-VAE commitment loss

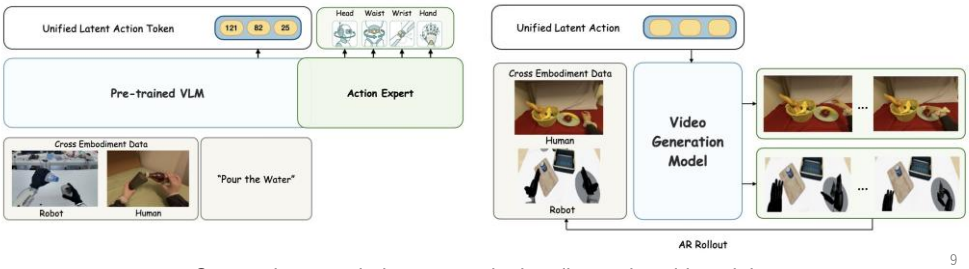
8

[E] Here is the core of the paper, and it is conceptually elegant. Look at the loss. For every branch — visual, action, and fusion — both decoders are used. That means: action features are required to predict the future visual outcome, and visual features are required to reconstruct the action. Why does this work? (Forcing action features to predict visual consequences) anchors heterogeneous kinematics to the same pixel-space outcomes, so a human arm motion and a humanoid arm motion that produce the same visual effect end up at the same token. Conversely, requiring visual features to reconstruct actions strips away appearance noise that has no motor counterpart — lighting and texture cannot help reconstruct an action, so they are filtered out. What survives this two-way bottleneck is precisely the intersection: an embodiment-agnostic physical intent.

[J] ここがこの論文のいちばん美しいアイデアです。損失関数を見てください。Visual、Action、Fusion のどのブランチに対しても、両方のデコーダで再構成させているんです。つまり、アクションの特徴量から将来の視覚的結果を予測させ、視覚の特徴量からアクションを再構成させる。なぜこれが効くのか。アクション特徴量に視覚的結果を予測させることで、異なる身体性の運動学が同じピクセル空間の結果に錨付けされる。同じ視覚的効果を生む人間の腕の動きとヒューマノイドの腕の動きが、同じトークンに対応するようになる。逆方向に、視覚特徴量からアクションを再構成させると、運動とは無関係な見た目のノイズが排除されます。照明やテクスチャはアクションの再構成に役立たないので、ボトルネックで自動的に削ぎ落とされるわけです。この双方向のボトルネックを生き残るものこそが、まさに 2 つのモダリティの「交差」、つまり身体性に依らない物理的意図というわけです。

Downstream Use: VLA-UniT & WM-UniT

VLA-UniT (policy)	WM-UniT (world model)
Build on GR00T n1.5 + Qwen2.5-VL backbone	Built on NVIDIA Cosmos Predict 2.5
VLM Predict UniT tokens from image + instruction	Continuous pre-quantization features as conditioning
Lightweight flow-matching head → embodiment-specific actions	Replaces raw action conditioning



9

Same token vocabulary serves both policy and world model.

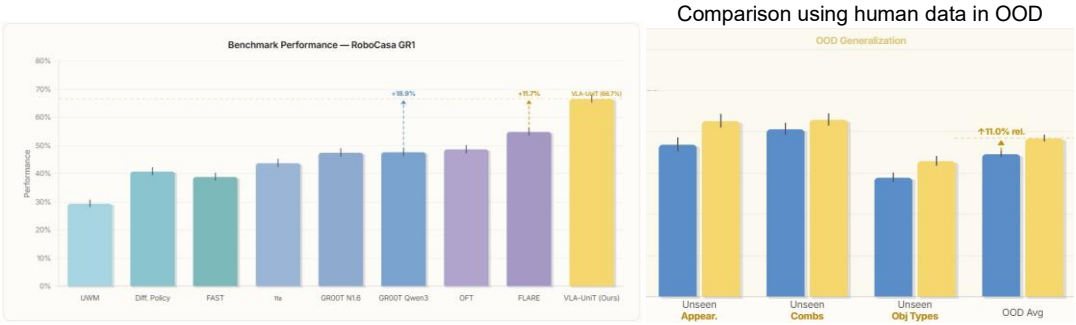
[E] The authors deploy UniT in two downstream paradigms. The first, VLA-UniT, is a policy. It is built on the GR00T n1.5 architecture with a Qwen2.5-VL backbone. The VLM takes the image and language instruction and predicts UniT tokens — the same tokens we just trained — and a lightweight flow-matching head turns those tokens into embodiment-specific continuous actions. The second, WM-UniT, is a world model. It uses the *continuous, pre-quantization* features of UniT as the conditioning signal for the Cosmos Predict 2.5 video generator, replacing raw action conditioning. The clean message here is that one tokenizer simultaneously serves both the action side and the world-modeling side.

[J] UniT トークンの使い道は 2 つ用意されています。1 つ目の VLA-UniT はポリシーで、GR00T n1.5 アーキテクチャ上に Qwen2.5-VL バックボーンを載せた構成です。VLM が画像と言語指示から UniT トークン、つまりさっき学習したのと同じトークンを予測し、軽量な flow-matching ヘッドがそのトークンを身体性ごとの連続アクションに変換します。2 つ目の WM-UniT はワールドモデルで、UniT の「量子化前の」連続特徴量を、Cosmos Predict 2.5 という動画生成モデルの条件付け信号として使います。生のアクションで条件付けるのを置き換えるわけです。要するに、ひとつのトークナイザがポリシーとワールドモデルの両方を同時に支えるという、きれいなメッセージになっています。

Results: Simulation & Few-Shot

- Outperforms UWM, Diffusion Policy, FLARE, GR00T-N1.6, π-Qwen3, FAST-Qwen3, OFT-Qwen3
- Co-training with EgoDex (human data): OOD 34.7% → 38.5%

Comparison using human data in OOD



[E] Let's look at simulation results on the RoboCasa GR1 benchmark, which covers 24 tabletop tasks. VLA-UniT reaches 66.7% average success rate, compared with 47.8% for the same architecture without UniT token prediction. That's an 18.9-point improvement that directly quantifies the contribution of UniT as a learning objective. And co-training with the EgoDex human video dataset further raises out-of-distribution success from 34.7% to 38.5%. The takeaway is that UniT helps most precisely where we need help — in low-data and OOD settings.

[J]シミュレーション結果を見ます。RoboCasa GR1 という 24 個のタスクからなるベンチマークです。VLA-UniT は平均成功率 66.7%。UniT トークン予測なしの同アーキテクチャ(GR00T-Qwen2.5)は 47.8%なので、18.9 ポイントの差です。これはまさに「UniT を学習目的に加えた効果」が定量化されたものと言えます。EgoDex という人間動画データセットとの共学習で OOD 成功率も 34.7%→38.5%に上がっています。一番嬉しい性質は、まさに「データが少ない・分布外」という困った場面で最も効くことです。

Results: Real-World & World Modeling

World modeling on DROID

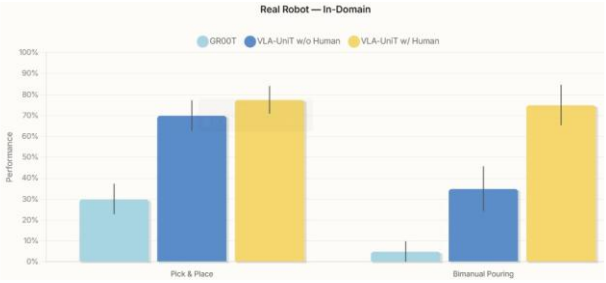
■ UniT-conditioned > raw-action / action-only-latent on PSNR, SSIM, LPIPS, FVD, EPE

Humanoid Robot: IRON-R01



Instruction: Put the banana in the box

METHOD	PSNR ↑	SSIM ↑	LPIPS ↓	FVD ↓	EPE ↓
Raw Action	21.02	0.820	0.097	**76.38**	0.2662
WM-Action	20.86	0.819	0.102	80.30	0.2593
WM-UniT	**21.32**	**0.823**	**0.095**	76.44	**0.2588**



[E] Moving to real-world experiments on the IRON-R01 humanoid. VLA-UniT reaches 78% on Pick & Place and 75% on bimanual pouring, well above the GR00T baseline. The most exciting result is on an unseen Stacking task: only the variant co-trained on human videos succeeds, at 60%, and the authors observe emergent whole-body behaviors like waist rotation and head turning — behaviors that were never demonstrated by the humanoid itself, only by humans. On the world-modeling side, UniT-conditioned generation on DROID beats both raw-action conditioning and action-only-latent conditioning across all standard video-quality metrics — PSNR, SSIM, LPIPS, FVD — and especially on the endpoint controllability metric EPE. So the same token vocabulary transfers both across embodiments and across the policy–versus–world-model boundary

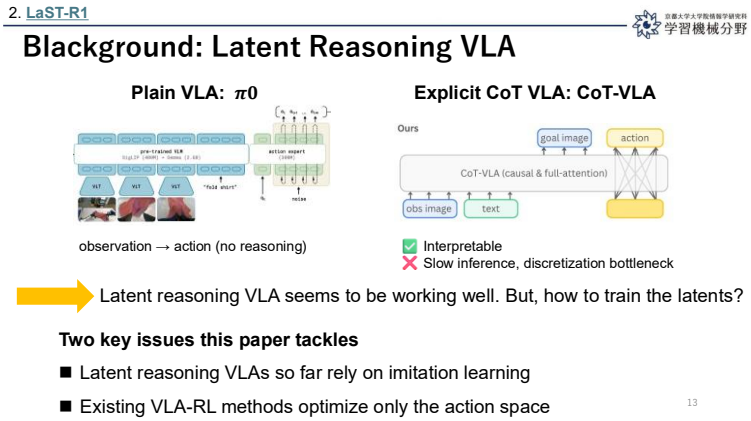
[J]実機実験は IRON-R01 ヒューマノイドで行われています。VLA-UniT は Pick & Place で 78%、両腕の注ぐタスクで 75%と、GR00T ベースラインを大きく上回っています。特に面白いのは未学習の Stacking タスクで、人間の動画と共学習したバリアントだけが 60%成功し、しかも腰の回転や頭の向きを変えるといった、ヒューマノイド自身は実演されていない、人間からしか観測できなかった全身協調動作が「創発」しています。ワールドモデル側では、DROID データセット上の UniT 条件付き生成が、生アクション条件付けやアクションのみの潜在条件付けを、PSNR、SSIM、LPIPS、FVD といった標準動画品質指標で全勝、特にコントローラビリティ指標 EPE で大差で勝っています。つまり、同じトークン語彙が「身体性をまたぐ転移」と「ポリシー↔ワールドモデルをまたぐ転移」の両方で機能している。

A Survey of Recent Advances in VLA

1. UniT: Toward a Unified Physical Language for Human-to-Humanoid Policy Learning and World Modeling
   B. Chen et al., “UniT: Toward a unified physical language for human-to-humanoid policy learning and world modeling.”
2. **LaST-R1: Reinforcing Robotic Manipulation via Adaptive Physical Latent Reasoning**
   H. Chen et al., “Last-r1: Reinforcing robotic manipulation via adaptive physical latent reasoning,” 2026. [Online]. Available: https://arxiv.org/abs/2604.28192
3. Streaming Flow Policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories
   S. Jiang et al., "Streaming Flow Policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories," in 9th Annual Conference on Robot Learning, CoRL 2025, 2025.

[E] Paper 2 is LaST-R1 — the first RL post-training framework for VLA models that *reason before acting*. It jointly optimizes the latent reasoning process and the action generation. Three keywords: Latent Chain-of-Thought, LAPO (Latent-to-Action Policy Optimization), and adaptive reasoning length. Before the technical details, the next slide covers some background on what "latent reasoning VLA" means.

[J] 2 本目、LaST-R1 です。「行動前に推論する」タイプの VLA に対する、初の強化学習 Post-training フレームワークで、潜在推論プロセスとアクション生成を同時に最適化します。キーワードは 3 つ、Latent Chain-of-Thought、LAPO（Latent-to-Action Policy Optimization）、適応的推論長です。技術の中身に入る前に、次のスライドで「潜在推論 VLA」とは何かという背景を整理します。



[E] Some background first. VLAs have evolved through three generations of reasoning. First, plain VLAs mapping observation directly to action with no reasoning. Second, explicit chain-of-thought VLAs that generate text or image plans before acting — interpretable, but slow and constrained by discretization. Third, the recent trend: latent reasoning VLAs, which produce a sequence of continuous latent vectors as their "thoughts" before generating actions. This is like LLM chain-of-thought, but in a continuous feature space, capturing fine-grained physical dynamics that text can't express.

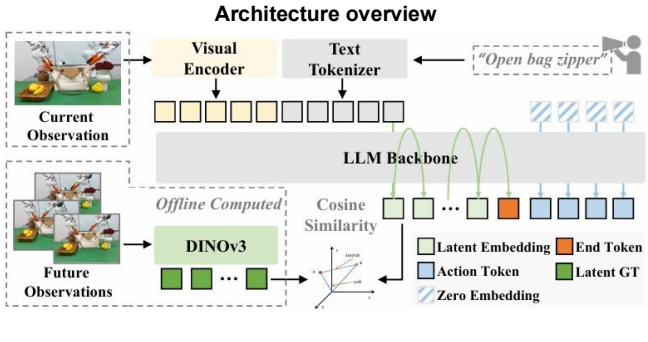
The key question is: *how do you train those latent vectors?* We'll come back to that in two slides. For now, note two problems this paper tackles. Existing latent reasoning VLAs rely purely on imitation learning, which needs huge expert datasets and breaks down under compounding errors. And existing VLA-RL methods like SimpleVLA-RL or πRL only optimize the action space — they bypass the model's internal reasoning entirely. LaST-R1 fills this gap by bringing RL to latent reasoning VLAs.

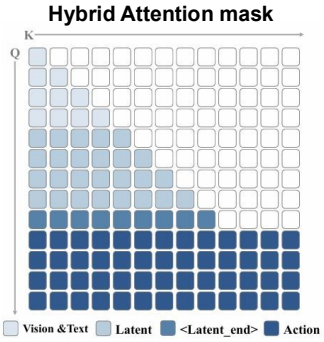
[J]まず背景です。VLA の「推論」は大きく 3 世代を経て進化してきました。第 1 世代は観測からアクションを直接マッピングする素朴な VLA。第 2 世代は行動前にテキストや画像プランを明示的に生成する Explicit CoT VLA — 解釈可能だが遅く、離散化の制約も大きい。そして第 3 世代が最近のトレンド、潜在推論 VLA で、「思考」として連続な潜在ベクトル列を生成してから行動する。LLM の CoT の類似物ですが、連続な特徴空間での思考なので、テキストでは表現できない細粒度の物理動態を捉えられる。

重要な問いが「その潜在ベクトルをどう訓練するか」で、これは 2 スライド後に戻ってきます。今は本論文が取り組む 2 つの問題に注目してください。既存の潜在推論 VLA は全て模倣学習に依存していて、巨大なエキスパートデータが必要で、誤差蓄積で破綻する。そして既存の VLA-RL 手法（SimpleVLA-RL、πRL など）はアクション空間しか最適化せず、内部推論を完全にバイパスしている。LaST-R1 はこのギャップを埋める、つまり潜在推論 VLA に RL を持ち込む論文です。

Architecture

1. Image + instruction → Qwen3-VL-4B + SigLIP2-Large encoder
2. LLM autoregressively emits latent embeddings (Terminated by <latent_end> token)
3. Parallel action decoding
4. Value head (4-layer MLP, shared backbone) for advantage estimation



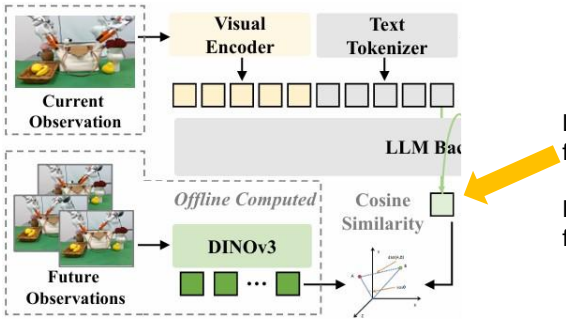


[E] The architecture, concretely. It's built on Qwen3-VL-4B with a SigLIP2-Large visual encoder. The backbone first autoregressively produces N_z continuous latent reasoning embeddings, terminated by a special `<latent_end>` token. Then actions are produced *in parallel* in a single forward pass — chunk size 8, 56 action tokens. A 4-layer MLP value head sharing the backbone handles advantage estimation. The trick making this efficient is the hybrid attention mask on the right: causal for the vision-text-latent prefix, bidirectional for the action tokens. So you get LLM-style sequential reasoning *and* parallel action speed in one model.

[J]アーキテクチャを具体的に。Qwen3-VL-4B に SigLIP2-Large の視覚エンコーダを載せた構成です。バックボーンはまず N_z個の連続な潜在推論埋め込みを自己回帰的に生成し、<latent_end>トークンで終端します。その後、アクションを「並列に」、1 回の forward pass で生成。チャンクサイズ 8、計 56 トークンです。アドバンテージ推定にはバックボーン共有の 4 層 MLP の value head を使用。効率を支えているのが右のハイブリッドアテンションマスクで、Vision+Text+Latent 部分は因果マスクで自己回帰、<latent_end>以降は双方向マスクで並列。LLM 流の逐次推論とアクションの並列デコードを、1 つのモデルで両立しています。

Training the Latents: What Should They Predict?

- Latents are continuous vectors. What should they be supervised against?
- Without a target, latents are just unconstrained noise. We need a physically meaningful learning signal.



Latent $z_{t,i}$ predicts the DINOv3 <CLS> feature of the future observation at $t+i$

DINOv3: SOTA self-supervised vision foundation model

[E] Back to the question we deferred: how do we *train* the latent reasoning? The latents are continuous vectors — without a target, they're just unconstrained noise. LaST-R1's answer is elegant: each latent $z_{t,i}$ is trained to predict the DINOv3 ⟨CLS⟩ feature of the *future observation* at time $t+i$. DINOv3 is a SOTA self-supervised vision foundation model, so its CLS token carries rich semantic and spatial features. Top-k channel selection with k=2560 matches the VLA's hidden size.

Why does this work? It forces the latents to *encode predicted future states* — literally "thinking about what will happen next."

[J]先送りした問いに戻ります。潜在推論をどう「訓練」するか。潜在ベクトルは連続なので、教師信号がなければただのノイズ。LaST-R1 の答えはエレガントで、各潜在 $z_{t,i}$を、時刻 $t+i$の「未来観測」の DINOv3 ⟨CLS⟩特徴量を予測するように訓練する。DINOv3 は最新の自己教師あり視覚基盤モデルで、CLS トークンには豊かな意味的・空間的情報が詰まっています。チャネル方向の top-k（k=2560）で VLA の隠れ次元と一致させる。

なぜ効くのか。潜在が「予測される未来状態」を符号化することを強制している、文字通り「次に何が起こるかを考える」ようになる。

LAPO: PPO on Latents + Actions

- Normal PPO: Positive advantage → push probability up.

  Maximize $r_t(\theta)\,\hat{A}_t$ $where$ $r_t = \pi_\theta / \pi_{\theta_{old}}$ (probability ratio)

- Action ratio (categorical, standard):

$$r_t^a(\theta) = \exp\left(log\pi_\theta(C_t) - log\pi_{\theta_{old}}(C_t)\right)$$

- Latent ratio (isotropic Gaussian):

$$r_t^z(\theta) = \exp\left(-\frac{1}{2\sigma^2}\sum_{i=1}^{N_z}\left\|z_{t,i}^{old} - z_{t,i}^{\theta}\right\|^2\right)$$

When $\hat{A}^t > 0$, pull latents toward good manifold of successful trajectories

16

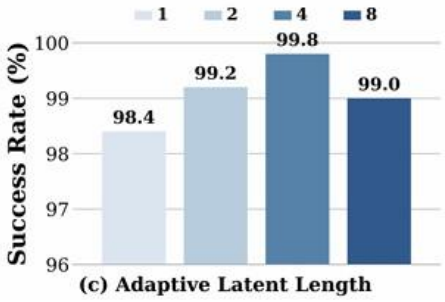
[E] The core algorithmic contribution: LAPO. Quick PPO recap on the top right — PPO maximizes the probability ratio between new and old policies, weighted by the advantage. Positive advantage pushes probability up, clipping prevents updates that are too large.
LAPO extends this to the latent space. For discrete actions, standard categorical ratios. For continuous latents, LAPO approximates the rollout distribution by an isotropic Gaussian centered at the current policy output. The derivation is elegant: the old policy's mean is by definition the sampled latent itself, so the old density at its own sample is just the normalization constant, which cancels. What's left is a clean closed form — exponential of negative-one-over-two-sigma-squared times the sum of squared latent distances. The clipped surrogate is then formed jointly over both ratios. Intuitively, when advantage is positive, the optimization pulls the current latents toward the "good-reasoning" manifold that produced successful trajectories.
[J]アルゴリズム面の中核が LAPO です。右上に PPO の簡単な復習 — 新旧ポリシーの確率比をアドバンテージで重み付けして最大化、アドバンテージが正なら確率を上げ、clipping で更新が大きすぎないようにする手法です。
LAPO はこれを潜在空間に拡張します。離散アクションは標準のカテゴリカル比、連続な潜在には、ロールアウト分布を「現在のポリシー出力を中心とする等方 Gaussian」で近似する。導出が綺麗で、旧ポリシーの平均は定義上「サンプリングされた潜在変数そのもの」なので、旧密度をその自分自身のサンプルで評価すると正規化定数になり、相殺される。残るのは $-1/(2\,\sigma^2)$ ×潜在距離の二乗和を指数の肩に乗せたきれいな閉形式。Clipped Surrogate を両方の比に対して同時に構成します。直感的には、アドバンテージが正のとき、潜在が「成功軌道を生んだ良い推論の多様体」に引き寄せられる、ということです。

Adaptive Latent CoT

- The length of the Latent CoT changes adaptively
- Inference: emit <latent_end> token, when $p \geq 0.99$
- Rollout: sample length from temperature-controlled categorical

$$p_m = \frac{\exp(l_m/\beta)}{\sum_{i=1}^{M} \exp(l_i/\beta)}$$

m : sampling length
M : max length



$M = 4$ is best!

[E] The adaptive latent CoT mechanism. Not every state needs the same deliberation depth — easy steps should be fast, hard steps should think longer. Within a max length of 8, <latent_end> can only be emitted at M predefined positions. At inference, exit as soon as the exit probability exceeds 0.99. During rollouts, lengths are sampled from a temperature-controlled categorical, with a separate transition-token loss teaching *when* to exit. The ablation in Figure 4(c) is interesting: M=1 fixed-length gives 98.4%, M=4 gives the best 99.8%, but M=8 — exit anywhere — drops back to 99.0%. Too much flexibility hurts; M=4 is the sweet spot.

[J]適応的な Latent CoT 機構です。全ての状態が同じ深さの思考を必要とするわけではない、簡単な場面はサッと、難しい場面はじっくり、と。最大長は 8、<latent_end>は決まった M 個の位置でしか出せない。推論時は終了確率が 0.99 を超えたら抜けます。ロールアウト中は候補長を温度付きカテゴリカル分布からサンプリングし、終了トークン専用の損失で「いつ抜けるか」を学習。図 4(c)の ablation が面白くて、M=1（固定長）98.4%、M=4 が最良の 99.8%、M=8（どこでも抜けられる）にすると 99.0%まで戻る。柔軟性を与えすぎても駄目で、M=4 がスイートスポットです。

Results: Simulation + Real-Wold

■ Comparison on the LIBERO (Sim)

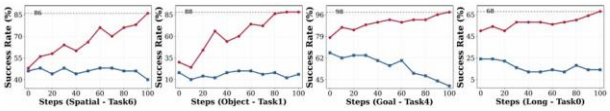
Models	Paradigm	Spatial		Object		Goal		Long		Average	
		SR↑	Rank↓	SR↑	Rank↓	SR↑	Rank↓	SR↑	Rank↓	SR↑	Rank↓
OpenVLA-OFT [5]	SFT	97.6	6	98.4	6	97.9	6	94.5	2	97.1	4
π_{RL}† [23]	RL	99.6	2	**100.0**	1	99.6	2	94.0	3	98.3	2
LaST-R1 (Ours)	RL	**99.8**	1	**100.0**	1	**100.0**	1	**99.8**	1	**99.9**	1

SFT
Prev SOTA

■ Comparison on Real-Wold Franka

Methods	Insert hexagon block				Open bag zipper			
	Original	Unseen-Object	-Background	-Lighting	Original	Unseen-Object	-Background	-Lighting
$\pi_{0.5}$ [7] (Full-size SFT)	65	35 (-30%)	55 (-10%)	40 (-25%)	75	30 (-45%)	70 (-5%)	60 (-15%)
LaST-R1 (Few-shot SFT → RL)	45 → 90	75 (-15%)	85 (-5%)	80 (-10%)	55 → 95	80 (-15%)	95 (-0%)	90 (-5%)

■ LAPO vs Action-Only PPO baseline



[E] LIBERO simulation. The data regime is harsh — only one-shot SFT warm-up plus online RL. LaST-R1 reaches 99.9% average, ranking first on every suite. The gap with the previous RL SOTA πRL is most pronounced on LIBERO-Long — 5.8 points — where long-horizon tasks let latent CoT shine. LaST-R1 even beats SFT baselines trained on the full 50-trajectory dataset.

real-world Franka R3, one single-arm and three dual-arm tasks. With only 30 trajectories per task and LoRA rank-32 updates during RL, LaST-R1 jumps from 52.5% to 93.75% average, beating full-SFT π-zero-point-five by 22.5 points. Bottle cap goes 45% to 95% — a 50-point gain.

The full-SFT baseline collapses under unseen objects, dropping up to 45 points. LaST-R1 stays within 15 points everywhere, with near-zero degradation against background or lighting changes — RL on latents forces the model to learn transferable physical concepts rather than memorize patterns.

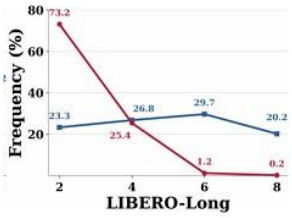
[J] LIBERO シミュレーション。データ条件は厳しく、SFT warm-up はワンショット+オンライン RL です。それでも LaST-R1 は平均 99.9%、全スイート 1 位。前 SOTA の πRL との差は LIBERO-Long で最も顕著で 5.8 ポイント差 — 長期視野タスクこそ Latent CoT の本領発揮です。フルデータで訓練された SFT ベースラインも上回っています。

Franka R3 での実機、単腕 1+両腕 3 の計 4 タスク。各タスク 30 軌道、RL 中は LoRA rank 32 のみ更新で、LaST-R1 は 52.5%→93.75%まで上昇、フル SFT の $\pi_{0.5}$を 22.5 ポイント上回ります。Bottle cap は 45%→95%の 50 ポイント増です。

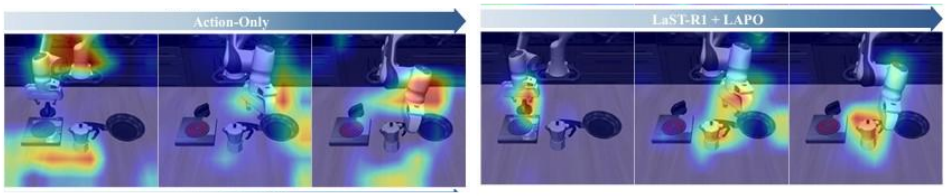
フル SFT ベースラインは未学習物体で最大 45 ポイント落ちて崩壊するのに対し、LaST-R1 は全条件で 15 ポイント以内、背景・照明変化に対してはほぼ無劣化 — 潜在空間での RL が、パターンの暗記ではなく転移可能な物理概念の学習を強制するから、というのが著者らの解釈です。

What did LaST-R1 actually lean?

- Reasoning length M shifts
  - Before RL: uniform over {2, 4, 6, 8}
  - After LAPO: sharp peaks at 2 and 4 → learns to think short



- Attention Map
  - Action-Only: diffuse, lags gripper
  - LaST-R1 + LAPO: focused, anticipates target



LIBERO-Long: pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate.

[E] the reasoning-length distribution shifts dramatically after LAPO — before RL it's roughly uniform over 2, 4, 6, 8, but after LAPO it peaks sharply at lengths 2 and 4. The policy learns to think *short* by default. Grad-CAM attention maps show LaST-R1 + LAPO develops focused, goal-directed attention that dynamically shifts from the manipulated object to the target. The takeaway: RL on latents is effectively *test-time scaling for robotics* — adaptive thinking budgets, shorter execution, and better attention, all learned automatically from reward.

[J] 推論長の分布が LAPO 後に劇的に変化。RL 前は長さ 2, 4, 6, 8 にほぼ一様分布が、LAPO 後は長さ 2 と 4 にシャープなピーク。デフォルトで「短く考える」ようになっています。

Grad-CAM で、LaST-R1+LAPO は焦点が定まった目標指向的な注意を持ち、操作物体→目標へと動的にシフトする。総括すると、潜在空間での RL は「ロボティクスの test-time scaling」として機能していて、適応的な思考予算、短い実行、優れた注意、すべてが報酬から自動で学ばれている、ということです。

A Survey of Recent Advances in VLA

1. UniT: Toward a Unified Physical Language for Human-to-Humanoid Policy Learning and World Modeling

   B. Chen et al., “UniT: Toward a unified physical language for human-to-humanoid policy learning and world modeling.”

2. LaST-R1: Reinforcing Robotic Manipulation via Adaptive Physical Latent Reasoning

   H. Chen et al., “Last-r1: Reinforcing robotic manipulation via adaptive physical latent reasoning,” 2026. [Online]. Available: https://arxiv.org/abs/2604.28192

3. **Streaming Flow Policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories**

   S. Jiang et al., "Streaming Flow Policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories," in 9th Annual Conference on Robot Learning, CoRL 2025, 2025.

[E] The last paper is Streaming Flow Policy — SFP — from MIT, CoRL 2025. The one-liner: SFP redesigns flow-matching policies so that flow time *coincides* with execution time, allowing actions to be streamed to the robot on the fly — an order of magnitude faster than diffusion policies, while keeping multi-modal behavior. Three keywords: action trajectory equals flow trajectory, a stabilizing velocity field, and about 4.5 milliseconds per action.

[J]最後の論文は、MIT の Streaming Flow Policy、略して SFP。CoRL 2025 発表です。1 行で言うと、「flow time と実行時間を一致させる」ことで、アクションをロボットに流れるように送り出せるようにしたもの。マルチモーダル性は保ったまま、Diffusion ポリシーより一桁速い。キーワードは 3 つ、Action trajectory = Flow trajectory、安定化付き速度場、アクション 1 つあたり約 4.5 ミリ秒です。

Background: Why Diffusion/Flow Policies are slow

Good point of Diffusion/Flow Policies

- Multi-modal action distributions (MSE-trained policies average over them)

Bad point of Diffusion/Flow Policies

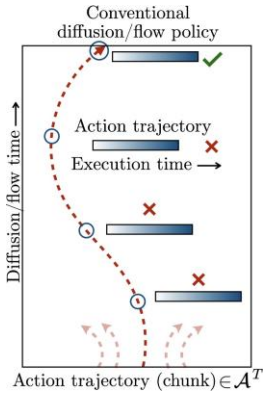
- Sample in action-sequence space A^T, No actions can reach the robot until the loop completes
- Start from pure Gaussian noise → must finish full denoising loop
- All intermediate trajectories are discarded

[E] Why have diffusion/flow policies become popular? Because they naturally represent multi-modal action distributions. For tasks like picking up a cup, several trajectories are equally valid; MSE-trained policies average them into degenerate behaviors, while diffusion preserves the modes. But the cost is severe: conventional policies sample in the action-sequence space $\mathcal{A}^T$ —a "trajectory of trajectories" — starting from pure Gaussian noise. All intermediate trajectories are discarded, and no action can be sent to the robot until the full denoising loop completes. Streaming Diffusion Policy partially fixes this with a rolling buffer of increasing noise, but it requires upfront buffer initialization.

[J] Diffusion/Flow ポリシーが人気な理由は、マルチモーダルなアクション分布を自然に表現できるから。コップを掴むようなタスクでは妥当な軌道が複数あって、MSE 訓練だと平均化されますが Diffusion はモードを保てる。しかし代償が大きい。従来手法はアクション系列空間 $\mathcal{A}^T$でサンプリングする「軌道の軌道」を生成していて、しかも純粋な Gaussian ノイズから始める。途中の軌道は全部捨てられるし、denoising が終わるまでロボットには何も送れない。Streaming Diffusion Policy（Høeg ら, 2025）は rolling buffer で一部緩和しますが、buffer の初期化に前払いコストが必要です。

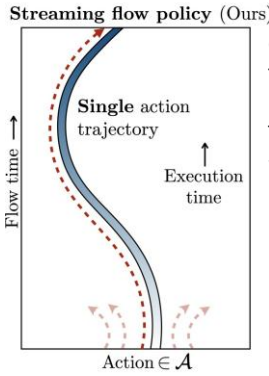
Key Idea: Action Trajectory=Flow Trajectory

■ Don’t sample trajectories, flow through one.



Generate Trajectory of trajectories

Must climb the full vertical axis before sending anything.



Generate a single action trajectory

flow time = execution time. Stream as you integrate.

22

[E] The central idea is beautifully simple. On the left, the conventional setup: vertical axis is flow time, horizontal is execution time, and you must climb the full vertical axis before sending any action. On the right, SFP: just *one* trajectory in action space, with flow time aligned to execution time. As the ODE solver integrates forward in flow time, it moves forward in execution time too, so each newly integrated action streams directly to the robot. The slogan: don't sample trajectories — flow through one. As a side benefit, the network's input/output dimension shrinks from $\mathcal{A}^T$ to just $\mathcal{A}$, so training is faster and uses less GPU memory.

[J]中心アイデアはシンプル。左の従来手法は、縦軸 flow time、横軸実行時間で、縦軸を最後まで登り切らないとロボットに何も送れない。右の SFP は、アクション空間に 1 本だけ軌道があって、flow time が実行時間と一致している。ODE solver が flow time で前に進むと実行時間でも前に進むので、積分されたアクションがそのままロボットに流れる。スローガンは「軌道をサンプリングするな、1 本を流れろ」。副次的に、ネットワークの入出力次元が $\mathcal{A}^T$から $\mathcal{A}$に縮むので、訓練も速くなり GPU メモリも減ります。

Method: Stabilizing Velocity Field

■ Construction (per demo ξ):

$$v_\xi(a,t) = \dot{\xi}(t) - k\big(a - \xi(t)\big)$$

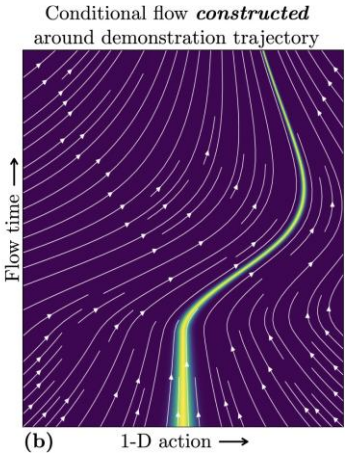
Trajectory velocity | Stabilization (P-control)

$$p_\xi^0(a) = N\big(a \big| \xi(0), \sigma_0^2\big)$$

initial position distribution

Resulting marginal

$$p_\xi(a|t) = N\big(a \big| \xi(t), \sigma_0^2 e^{-2kt}\big)$$



Theoretical motivation: Block et al. (NeurIPS 2023) stabilizing low-level controllers reduce distribution shift and tighten imitation learning bounds

[E] For each demonstration ξ, the authors analytically construct a stabilizing velocity field. The first term, ξ-dot, is the velocity along the demo. The second term, minus k times (a − ξ(t)), is a P-controller pulling deviations back. Theorem 1 shows the induced marginal is a Gaussian tube around the demo, with width shrinking as e^{-2kt} .Two key remarks: first, SFP exploits the time derivative $\dot{\xi}(t)$,which conventional diffusion/flow policies don't use. Second, the stabilization has theoretical backing — Block et al. at NeurIPS 2023 showed that stabilizing controllers reduce distribution shift in imitation learning.

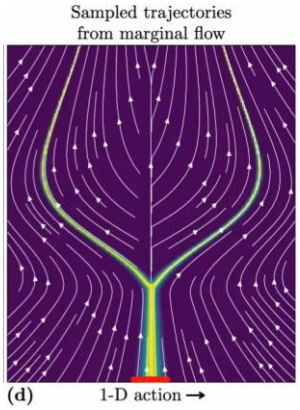
[J]各実演 ξ に対し、安定化付きの速度場を解析的に構成します。第 1 項 $\dot{\xi}(t)$は実演に沿った速度、第 2 項 $-k\big(a-\xi(t)\big)$は P 制御で偏差を引き戻す項。Theorem 1 により、誘導される周辺分布は実演周りの Gaussian チューブで、幅が e^{-2kt}で縮みます。要点が 2 つ。1 つ目、SFP は時間微分 $\dot{\xi}(t)$を使っていて、これは従来の Diffusion/Flow ポリシーが使わない情報。2 つ目、安定化には理論的根拠があり、NeurIPS 2023 の Block らが「安定化制御器は模倣学習で分布シフトを減らす」ことを示しています。

Training & Inference

- Training (flow matching loss):

$$L_{CFM}(v, pD) = E_{(h,\xi)\sim pD, t\sim U[0,1], a\sim p\xi(a|t)} \| v(a, t \mid h) - v\xi(a, t) \|_2^2$$

- Inference
  - At test time, set $\sigma_0 = 0$ → deterministic policy
  - ODE solver integrates autoregressively: each action depends only on previously computed ones
  - Each newly integrated action → streamed directly to controller



Initialize integration from measured robot state, proprioceptive feedback enters via both h and a_0.
→ reduces accumulated error

[E] Training is standard conditional flow matching: regress the network onto the analytical velocity field. By Lipman et al.'s theorem, the minimizer's marginal at every t matches the training-data marginal, preserving multi-modality. At inference, $\sigma_0$is set to zero — multi-modality lives in the network weights, not in test-time noise, the same trick ACT uses. The ODE solver integrates autoregressively, so each new action streams to the controller immediately. There's also a clever extension called "state imitation": initialize integration from the *measured* robot state, so proprioception enters the policy through both the observation history h and the initial action a_0 ,reducing accumulated error.

[J]訓練は標準的な条件付き flow matching で、ネットを解析的な速度場に回帰させる。Lipman らの定理で、最適解の周辺分布が各時刻 t で訓練データと一致し、マルチモーダル性が保たれます。推論時は $\sigma_0 = 0$にして決定的なポリシーにする。マルチモーダル性は重みに入っているので、テスト時のノイズは不要、ACT と同じ発想です。ODE solver は自己回帰的に積分するので、新しいアクションは即座にコントローラへ流せる。さらに「state imitation」という拡張があり、積分の初期値を実測ロボット状態から始める。すると、proprioception が観測履歴 hと初期アクション a_0の両方からポリシーに入って、累積誤差を減らせます。

Results: Simulation

■ Push-T

		Push-T with state input			Push-T with image input	
		State imitation Avg/Max scores ↑	Action imitation Avg/Max scores ↑	Latency ↓	State imitation Avg/Max scores ↑	Latency ↓
1	DP [1]: 100 DDPM steps	92.9% / 94.4%	90.7% / 92.8%	40.2 ms	**87.0%** / 90.1%	127.2 ms
2	DP [1]: 10 DDIM steps	87.0% / 89.0%	81.4% / 85.3%	04.4 ms	85.3% / **91.5%**	10.4 ms
3	Flow matching policy [5]	80.6% / 82.6%	80.6% / 82.6%	05.8 ms	71.0% / 72.0%	12.9 ms
4	Streaming DP [14]	87.5% / 91.4%	84.2% / 87.0%	26.7 ms	84.7% / 87.1%	77.7 ms
5	SFP *without* stabilization	84.0% / 86.4%	81.8% / 93.2%	**03.5 ms**	73.9% / 77.5%	**08.8 ms**
6	**SFP (Ours)**	**95.1% / 96.0%**	**91.7% / 93.7%**	**03.5 ms**	83.9% / 84.8%	**08.8 ms**

$$v_\xi(a,t) = \dot{\xi}(t) - k\big(a - \xi(t)\big)$$

P-controller term is important!

■ RoboMimic



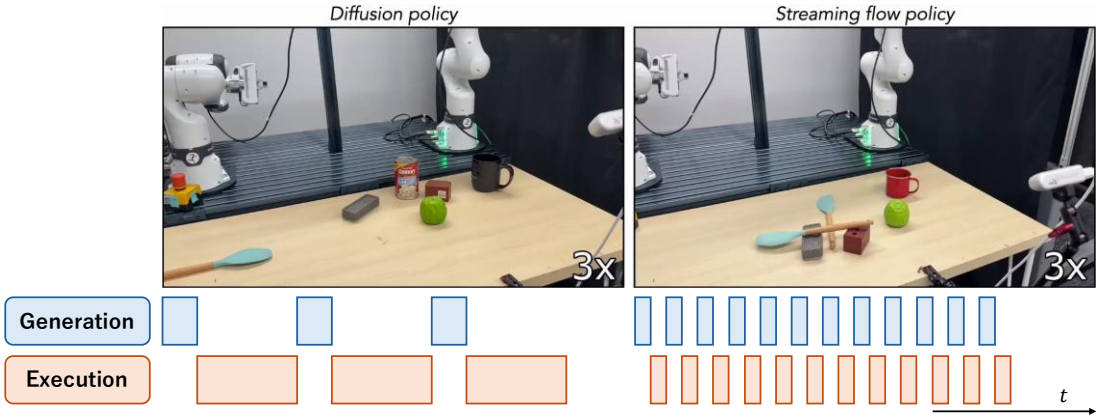
		RoboMimic Lift Action imitation Avg/Max scores ↑	RoboMimic Can Action imitation Avg/Max scores ↑	RoboMimic Square Action imitation Avg/Max scores ↑	Latency ↓
1	DP [1]: 100 DDPM steps	**100.0% / 100.0%**	94.0% / 98.0%	77.2% / **84.0%**	53.4 ms
2	DP [1]: 10 DDIM steps	**100.0% / 100.0%**	91.8% / 98.0%	76.0% / 82.0%	5.8 ms
3	Flow matching policy [5]	99.2% / **100.0%**	66.0% / 80.0%	54.0% / 56.0%	4.8 ms
4	Streaming DP [14]	98.8% / **100.0%**	96.8% / 98.0%	77.6% / 82.0%	30.3 ms
5	SFP *without* stabilization	99.6% / **100.0%**	90.0% / 92.0%	53.2% / 60.0%	**4.5 ms**
6	**SFP (Ours)**	**100.0% / 100.0%**	**98.4% / 100.0%**	**78.0% / 84.0%**	**4.5 ms**

[E] The numbers. Push-T state input: SFP 95.1/96.0, beating DDPM-100's 92.9/94.4 and every baseline. RoboMimic Can: 98.4/100, best. Square, the hardest task: SFP ties DDPM-100 at 78.0/84.0 — but the faster baselines like 10-step DDIM and flow-matching trade significant accuracy for speed. On latency, SFP runs at 3.5–4.5 ms per action, an order of magnitude faster than DDPM-100. The ablation confirms stabilization is essential: setting k to zero drops Square from 78.0 to 53.2, and Can from 98.4 to 90.0. Importantly, these latency numbers exclude the additional speedup from running generation and execution in parallel — the real wall-clock benefit is even larger.

[J]数字を見ていきます。Push-T 状態入力で SFP は 95.1/96.0%、DDPM-100 の 92.9/94.4%や他のベースラインを上回る。RoboMimic Can は 98.4/100%で最高。最難関の Square は 78.0/84.0%で DDPM-100 と同等。ただし 10 ステップ DDIM や純粋な flow-matching は速度のために精度を大きく犠牲にしている。レイテンシは SFP が 3.5〜4.5ms、DDPM-100 の一桁速い。アブレーションからも安定化項が必須で、$k = 0$ にすると Square が 78.0→53.2%、Can が 98.4→90.0%まで落ちます。重要な点として、このレイテンシは生成と実行の並列化による追加の高速化を含んでいないので、実際の体感優位はさらに大きいです。

Results: Real-World

- Tasks: Reach and pick an apple
- Settings: Franka Research 3 + RealSense D435f

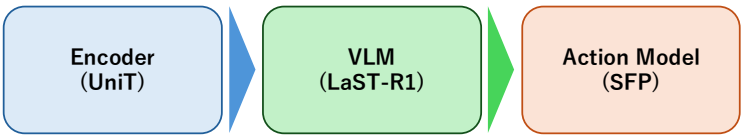


[E] Real-world experiments use a Franka Research 3 with a RealSense D435f, on apple-picking and block reorientation. SFP produces noticeably smoother and faster motion than Diffusion Policy. Here's the subtle point: the smoothness doesn't only come from lower average latency. SFP emits actions at a *uniform* interval as the ODE integrates, while diffusion policies finish the whole denoising loop before sending anything — producing a stop-and-go pattern even with similar average latency. Plus, SFP allows async parallel generation and execution for an even tighter loop. So uniform emission and async overlap are real advantages on top of raw speed.

[J]実機実験は Franka Research 3 と RealSense D435f で、リンゴ取りとブロック再配置の 2 タスク。SFP は Diffusion Policy より明らかに滑らかで速い。ここがポイントですが、滑らかさは平均レイテンシだけから来るのではない。SFP は ODE 積分中、アクションを均等な間隔で出し続けるのに対し、Diffusion 系は denoising が終わるまで何も送れず、平均レイテンシが似ていても stop-and-go 型になる。さらに SFP は生成と実行の非同期並列化が可能なので、ループをもっと締められる。つまり「均等な出力」と「非同期並列」が、純粋な速度とは独立した利点になっているわけです。

Toward a VLA that behaves more human-like

- RL will become increasingly important
  - LaST-R1 demonstrated that a method of simultaneously RL Action Model and VLM is effective.
  - SFP has the potential to integrate more naturally into RL because each inference corresponds to one Step of Action.
- Behavioral expressions that are independent of their embodiment are important as a way of expressing human behavioral intentions (UniT).



27

And that wraps up the three papers — the common thread across them is tightening the loop between the learned policy and the physical world.

Before I close, let me briefly connect these papers to my own research perspective. I believe reinforcement learning will become increasingly important going forward. LaST-R1 has already demonstrated that jointly RL-training an action model and a VLM is highly effective. From this perspective, SFP looks particularly promising — because each inference step corresponds to a single action step, SFP can integrate into an RL framework more naturally than conventional diffusion or flow policies, where the "trajectory of trajectories" structure makes step-level credit assignment awkward.

At the same time, looking at UniT, I'm convinced that embodiment-independent behavioral representations are essential as a way of expressing human behavioral intent. Decoupling *what* the human wants to do from *which* robot executes it seems to me a key direction for scalable imitation learning.

Taken together, I see a path forward that combines streaming, step-aligned policies like SFP with embodiment-independent intent representations like UniT, trained jointly with RL in the style of LaST-R1. That's the direction I'd like to explore.

Thank you very much for listening — happy to take questions.

以上で 3 本の論文紹介を終わります。共通テーマは「学習ポリシーと物理世界のループを締める」という方向性でした。

締めに、これらの論文と私自身の研究上の視点を少しだけ繋げさせてください。今後、強化学習はますます重要になっていくと考えています。LaST-R1 は、Action Model と VLM を同

時に RL で学習する手法が有効であることを既に示しました。この観点から見ると、SFP は特に有望です。なぜなら、1 回の推論が 1 ステップのアクションに対応するため、SFP は従来の Diffusion/Flow ポリシーよりも自然に RL の枠組みに統合できるからです。従来手法の「軌道の軌道」という構造では、ステップ単位の credit assignment が扱いにくいのに対し、SFP はその問題を回避できます。

同時に、UniT を見ていると、身体性に依存しない行動表現が、人間の行動意図を表現する方法として本質的に重要だと感じます。「人間が何をしたいか」と「どのロボットが実行するか」を切り離すことは、スケーラブルな模倣学習における鍵になる方向性だと思います。

これらを総合すると、SFP のような streaming・step-aligned なポリシーと、UniT のような身体性に依存しない意図表現を組み合わせ、LaST-R1 のように RL と協調学習させる、という方向性が見えてきます。これが私が今後探究していきたい方向です。

ご清聴ありがとうございました。質問をお受けします。